



Fast extremum graph computation for large-scale arbitrary grids

Quming Li^a, Zhengwen Liu^a, Zhibin Huang^{a,*}, Wantong Li^a, Zhiqiang Chu^b,
Zhitao Dai^a, Min Yu^c, Fang Deng^a

^a Beijing University of Posts and Telecommunications, Beijing, 100876, China

^b China Cybersecurity Review, Certification and Market Regulation Big Data Center, Beijing, 100086, China

^c China Academy of Aerospace Aerodynamics, Beijing, 100071, China

HIGHLIGHTS

- A novel fast saddle detection mechanism for large-scale arbitrary grids is designed.
- A universal parallel extremum graph construction framework for structured and unstructured grids of any dimension is proposed.
- The first GPU-accelerated extremum graph computation method is implemented within the TTK framework.

ARTICLE INFO

Editor: Rida T. Farouki

Keywords:

Extremum graph

Morse theory

GPU acceleration

Topological data analysis

ABSTRACT

The extremum graph is a compact topological representation designed to capture critical structures in scalar fields, and has been widely applied in flow field analysis, scientific visualization, and feature extraction. However, existing methods for constructing extremum graphs are typically restricted to structured grids and rely on exhaustive saddle point detection for every sample, which results in extremely low computational efficiency or significant precision degradation when large-scale unstructured data must be resampled onto structured grids. To overcome these limitations, we propose a fast extremum graph construction framework, denoted as *AnyEG*, which supports both structured and unstructured grids in arbitrary dimensions. The method integrates local path compression with gradient flow analysis, and introduces the concept of *boundary saddle points* to achieve efficient and accurate saddle point detection. Grounded in discrete Morse theory, our approach substantially reduces redundant computation and memory access overhead. Furthermore, to the best of our knowledge, *AnyEG* is the first GPU-based implementation of an extremum graph algorithm within the Topology ToolKit (TTK) framework. Comprehensive experiments on multiple structured and unstructured datasets demonstrate that our method achieves an average speedup of nearly five times over the state-of-the-art Tachyon method, while preserving complete topological correctness.

1. Introduction

Morse theory provides a rigorous mathematical foundation for describing the global topology of scalar fields (Morse, 1934). Building upon this foundation, the Morse–Smale Complex (MSC) accurately characterizes the global organization of gradient flows within a scalar field by partitioning the manifold into mutually disjoint stable and unstable regions associated with these flows

* Corresponding author.

Email address: huangzb@bupt.edu.cn (Z. Huang).

<https://doi.org/10.1016/j.cagd.2026.102519>

Received 4 September 2025; Received in revised form 7 November 2025; Accepted 13 February 2026

Available online 17 February 2026

0167-8396/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

(Thanh et al., 2024). However, the classical Morse theory and MSC formulations assume that the scalar field is continuous and differentiable, an assumption that is often difficult to satisfy directly in practical numerical simulations or empirical measurements.

Discrete Morse Theory extends classical Morse theory to discrete data structures while preserving topological consistency, thereby enabling its direct application to grid-based data for the extraction of gradient flows and the identification of critical points. This, in turn, facilitates the computational construction of the MSC (Ophelders et al., 2025).

The extremum graph can be viewed as a topological spine substructure within the MSC framework (Correa et al., 2011). It selectively retains extrema (maxima and minima) and their directly connected saddle points, establishing topological relationships through gradient flow paths. Compared to the full MSC structure, the extremum graph substantially reduces storage requirements while preserving the essential global skeleton of gradient flows.

Owing to their compactness, representational clarity, and low computational overhead, extremum graphs have been widely adopted in various application domains, including scalar field segmentation (Das et al., 2024; Tierny and Pascucci, 2012), flow field structure extraction (Weinkauff et al., 2010), feature tracking and event detection (Gyulassy et al., 2007), spatio-temporal data evolution analysis (Bremer et al., 2010), as well as topological clustering and dimensionality reduction for high-dimensional data (Liu et al., 2017). When dealing with large-scale, unstructured, noisy, or high-dimensional datasets, extremum graphs demonstrate superior scalability and robustness compared to the complete MSC, owing to their lightweight architecture and inherent topological stability.

With the continuous growth of high-resolution data, efficiently constructing extremum graphs while preserving topological accuracy has become a critical research problem in topological analysis. However, two major challenges currently hinder the construction of extremum graphs: (1) For scalar fields defined on unstructured grids, there is still no efficient method for extremum graph computation; (2) The determination of saddle points in scalar fields remains inefficient and is highly dependent on structured grids.

Tachyon (Ande et al., 2023) and ExTreeM (Lukasczyk et al., 2023) propose saddle point identification schemes based on Morse theory for structured grids. However, these methods require exhaustive point-by-point evaluations and heavily rely on structured grid regularity to simplify neighbor searching and connectivity analysis.

For unstructured grids, existing approaches can only perform explicit point-wise traversals, which are extremely inefficient. During traversal, it is necessary not only to identify all neighboring elements but also to determine their mutual connectivity. To date, there is no method that can efficiently construct extremum graphs for unstructured grids.

To address these challenges, this paper introduces *AnyEG*, a novel and efficient extremum graph construction framework designed for arbitrary grid types. The proposed method is grid-structure-agnostic, supporting both structured and unstructured grids. It employs a path compression strategy to rapidly identify the extremum control region associated with each vertex, thereby avoiding the explicit computation of global gradient flow paths.

In addition, a fast saddle point detection mechanism is developed, introducing the concept of *boundary saddle points* to further accelerate saddle identification. Although PLMSS (Maack et al., 2024) also computed region separators, it merely utilized the notion of regions without proposing a specific strategy for saddle point identification. In contrast, our method explicitly formulates a principled and efficient saddle point determination scheme, representing a novel contribution to extremum graph construction.

By leveraging discrete Morse theory for the topological refinement of boundary saddle points, *AnyEG* substantially reduces redundant saddle detection and memory access overhead. Furthermore, it provides the first GPU-accelerated parallel implementation within the Topology ToolKit (TTK) framework.

Finally, through persistence-based filtering of graph structures combined with arc bundling techniques, *AnyEG* constructs a sparse yet topologically accurate extremum graph.

The main contributions of this paper are summarized as follows:

- We propose, for the first time, a universal parallel framework for extremum graph construction, applicable to both structured and unstructured grids of arbitrary dimensions, without relying on structured grid assumptions;
- We design a novel fast saddle point detection mechanism that integrates flow path compression with field domain boundary analysis, substantially reducing computational complexity;
- We implement the first GPU-accelerated extremum graph computation method within the Topology ToolKit (TTK) framework, and validate it on a variety of structured and unstructured datasets. Experimental results demonstrate significant performance gains while maintaining topological correctness.

The remainder of this paper is organized as follows: Section 2 introduces the theoretical foundations and reviews the state-of-the-art related work; Section 3 details the proposed methodological framework and its key implementation aspects; Section 4 presents a comprehensive experimental evaluation, analyzing computational efficiency and topological accuracy across various scenarios; Section 5 discusses the limitations and scope of the proposed approach; and Section 6 concludes the paper and outlines directions for future research.

2. Background and related work

2.1. Gradient flow and gradient path

Extremum graphs characterize the global topological structure of scalar fields by establishing connections among distinct types of critical points (maxima, minima, and saddles), where gradient paths serve as the fundamental building blocks of these connections. Gradient paths trace the trajectories of points that follow the direction of the gradient within a scalar field. They enable the partitioning

of the data domain into basins of attraction associated with individual critical points and are widely used in multiscale analyses to track the formation and disappearance of topological structures.

The theory of gradient flows provides a unified analytical framework for constructing extremum graphs in both continuous and discrete settings. In continuous domains, gradient trajectories can be precisely determined, whereas in discrete domains, they are typically approximated through local neighborhood comparisons. This transition from local gradient behavior to global connectivity allows extremum graphs to capture not only the geometric characteristics of scalar fields but also the evolution of their topological structures across scales.

For the continuous case, consider a smooth scalar function

$$f : \Omega \rightarrow \mathbb{R}, \quad (1)$$

where Ω denotes a Euclidean space or a smooth manifold. The gradient flow of f is governed by the differential equation

$$\frac{dx}{dt} = \pm \nabla f(x), \quad (2)$$

whose solution curve characterizes the trajectory of ascent or descent along the gradient direction. Within the scalar field, the flow direction at each point is uniquely determined and eventually converges to a corresponding extremum, which serves as its attractor (Milnor, 1963; Edelsbrunner and Harer, 2010).

For the discrete case, let the scalar field be defined on a simplicial complex K as

$$g : V(K) \rightarrow \mathbb{R}, \quad (3)$$

where $V(K)$ denotes the set of vertices of K . Since continuous gradients cannot be directly defined on discrete structures, the gradient direction is typically approximated through local neighborhood comparisons. For any vertex $p \in V(K)$, its steepest-ascent and steepest-descent neighbors are defined as

$$\nabla^+ g(p) = \arg \max_{q \in N(p)} g(q), \quad \nabla^- g(p) = \arg \min_{q \in N(p)} g(q), \quad (4)$$

where $N(p)$ denotes the set of adjacent vertices of p .

Based on this definition, the steepest-ascent path can be constructed as

$$p \rightarrow \nabla^+ g(p) \rightarrow \nabla^+(\nabla^+ g(p)) \rightarrow \dots \rightarrow m, \quad (5)$$

which ultimately terminates at a local maximum m , satisfying $\nabla^+ g(m) = \emptyset$. Similarly, the steepest-descent path terminates at a local minimum. Together, these paths form the discrete gradient flow structure of the scalar field.

Discretization of local gradient direction. In a discrete grid or graph structure, the gradient direction can be approximated by comparing scalar values within a local neighborhood. For any vertex v , its discrete gradient direction is defined as

$$n^* = \arg \max_{n \in \mathcal{N}(v)} f(n), \quad (6)$$

where $\mathcal{N}(v)$ denotes the set of neighboring vertices of v . If $f(v) > f(n)$ for all $n \in \mathcal{N}(v)$, the vertex v is identified as a local maximum, corresponding to the discretized case in which the gradient vanishes.

The discrete representation of a gradient path corresponds to the traversal trajectory of the parent-pointer chain in our algorithm, which provides the theoretical foundation for region assignment and extremum structure partitioning. Further implementation details are discussed in Section 3.

2.2. Extremum graphs and morse theory

Extremum graphs provide a simplified yet informative graphical abstraction for describing the primary topological structures of scalar fields. They are particularly effective in representing the connections between extremal points established through integral lines. The theoretical foundation of extremum graphs originates from Morse theory, which investigates how smooth real-valued functions defined on manifolds reveal the topological structure of the underlying space through their critical points.

Let M be an n -dimensional compact smooth manifold without boundary, and let $f : M \rightarrow \mathbb{R}$ be a C^2 smooth real-valued function defined on M . If all critical points of f (i.e., points satisfying $\nabla f(p) = 0$) are non-degenerate—that is, the Hessian matrix $\text{Hess}_f(p)$ is non-singular at p —then f is called a *Morse function*. The index of each critical point p is determined by the number of negative eigenvalues of $\text{Hess}_f(p)$: an index of 0 corresponds to a minimum, an index of n corresponds to a maximum, and an index k with $0 < k < n$ corresponds to a k -saddle. According to discrete Morse theory, such critical points can be intuitively characterized by their upper-link and lower-link structures within the complex.

In discrete Morse theory, Forman (1998) proposed a framework that generalizes classical Morse theory to discrete spaces such as simplicial complexes and cell complexes. Instead of analyzing gradients and Hessians of smooth functions, discrete Morse theory characterizes “critical structures” through discrete functions defined on the complex (typically assigning scalar values to simplices). It further defines discrete critical simplices and their corresponding indices, providing a powerful tool to study topological properties such as homology and homotopy of discrete spaces.

In the discrete setting, the *upper link* and *lower link* of an n -dimensional simplex $\sigma^n \in K$ are defined as follows:

- **Lower link:** the set of all $(n-1)$ -dimensional faces τ^{n-1} of σ^n such that $f(\tau^{n-1}) > f(\sigma^n)$;
- **Upper link:** the set of all $(n+1)$ -dimensional cofaces η^{n+1} of σ^n such that $f(\eta^{n+1}) < f(\sigma^n)$.

From a more intuitive discrete perspective, the upper and lower links can be understood through the *ascending* and *descending* components associated with each vertex. Let v be a vertex in the complex with a scalar value $f(v)$, then:

- The **upper link** of v (denoted $Lk^+(v)$) consists of the adjacent vertices whose function values are greater than $f(v)$. These connected components in the local neighborhood are referred to as *ascending components*, and their connectivity number is denoted by $|Lk^+(v)|$;
- The **lower link** of v (denoted $Lk^-(v)$) consists of the adjacent vertices whose function values are less than $f(v)$. The corresponding connected components are referred to as *descending components*, and their connectivity number is denoted by $|Lk^-(v)|$.

This structure reflects the local gradient flow behavior of the function near v : the upper link represents the reachable paths in the direction of increasing function values, while the lower link represents those in the direction of decreasing values. Hence, the type of the vertex v can be determined by the number of connected components in its upper and lower links:

- If $|Lk^-(v)| = 1$ and $|Lk^+(v)| = 0$, there is no upstream path, and v is a *maximum*;
- If $|Lk^+(v)| = 1$ and $|Lk^-(v)| = 0$, there is no downstream path, and v is a *minimum*;
- If $|Lk^-(v)| = 1$ and $|Lk^+(v)| = 1$, the gradient structure undergoes no topological change, and v is a *regular point*;
- If $|Lk^-(v)| > 1$ or $|Lk^+(v)| > 1$, and v is not an extremum, the gradient structure exhibits a topological change at v , and v is a *saddle point*.

The above definition highlights that saddle points serve as crucial transition nodes connecting multiple extrema, thereby characterizing the topological relationships among extremal structures.

To illustrate this, consider the construction of a *maximum graph*. We begin with an unstructured grid composed of two-dimensional simplices (i.e., triangles). In practical applications, many grids are not purely triangular; however, for consistency, they can be converted into equivalent two-dimensional simplex grids.

Based on this setup, we use such an unstructured grid as an example to demonstrate how discrete Morse theory is applied in the construction of a maximal graph, as illustrated in Fig. 1.

For point 66 (Fig. 1a), there exists a lower-link path (blue) connecting point 45 to point 30 to point 40, with no upper link. Therefore, point 66 is classified as a *maximum*.

For point 37 (Fig. 1b), two upper-link paths (red) originate from points 99 and 75, and one lower-link path (blue) leads to point 9. Hence, point 37 is identified as a *saddle*.

For point 21 (Fig. 1c), there are two upper links (red) and two lower links (blue), both satisfying the saddle conditions; thus, point 21 is also a *saddle*.

For point 2 (Fig. 1d), exactly one upper link and one lower link are present, indicating that the gradient structure undergoes no topological change; consequently, point 2 is classified as a *regular point*.

This classification approach, which relies on the connectivity patterns of upper and lower links, is widely adopted in discrete topological analysis, including discrete Morse simplification, extremum graph construction, and topological visualization. It offers an effective means of identifying critical points that remains fully consistent with classical Morse theory.

2.3. Related work

Extremum graphs can be regarded as one-dimensional skeletons of Morse complexes, whose current canonical form was first proposed by Correa et al. (2011), together with an intuitive two-dimensional visualization scheme known as *topological spines*.

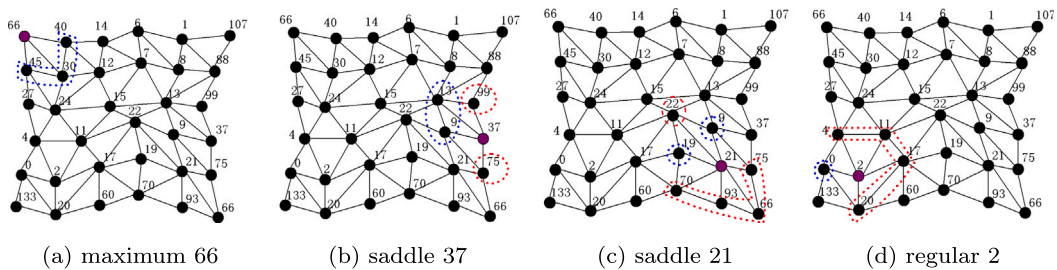


Fig. 1. Identification and classification of critical points (dark violet) in the maximum graph based on the connectivity of upper (red) and lower (blue) links. Specifically, maximum 66 has UL = 0 and LL = 1; saddle 37 has UL = 2 and LL = 1; saddle 21 has UL = 2 and LL = 2; and regular point 2 has UL = 1 and LL = 1. Here, UL and LL denote the numbers of upper and lower links, respectively. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

As the scale and complexity of scientific data continue to grow, research in topological visualization has increasingly focused on developing parallel algorithms to fully exploit modern high-performance computing architectures. Gyulassy et al. (2008) introduced a memory-efficient parallel algorithm for 3D Morse–Smale complex (MSC) computation by partitioning large-scale data into independent “data blocks” and merging the results in a post-processing stage. This approach enables topological extraction from extremely large datasets but is limited to structured grids, lacking unified modeling capabilities for unstructured domains. Peterka et al. (Peterka et al., 2011; Gyulassy et al., 2012) extended this method to distributed-memory cluster environments, yet their global communication strategy hinders efficient adaptation to GPU architectures and provides no native support for extremum graph construction. Shivashankar and Natarajan (Shivashankar et al., 2011; Bruckner et al., 2012) proposed a locally independent definition of gradient pairs and developed the *pym3d* library (Shivashankar and Natarajan, 2017), enabling parallel MSC processing on both CPUs and GPUs. However, their saddle detection scheme requires computing gradient paths for all points, which incurs substantial overhead on sparse or irregular meshes. Subhash et al. (2020, 2022) proposed the *gMSC* algorithm, which offloads the entire Morse structure computation to the GPU, achieving significant acceleration. Nevertheless, their method still focuses on merge tree construction and lacks specific optimization for extremum graph structures, as well as direct compatibility with visualization toolkits such as TTK. Ande et al. (2023) presented a shared-memory-based parallel method for extremum graph construction, achieving high runtime efficiency in multi-threaded CPU environments. However, it only supports regular grids, cannot handle unstructured data, and does not consider GPU acceleration. Das (2023) further proposed a boundary-management optimization strategy for path merging in extremum graphs, reducing redundant operations and improving robustness, yet their saddle extraction still relies on a global search, resulting in high computational costs. Maack et al. (2024) developed a well-scaling parallel algorithm for Morse–Smale segmentation, including region separators and boundaries; however, their approach focuses solely on regional decomposition and does not address saddle point identification in detail.

Beyond parallel computation, extremum graphs have also found broad applications in advanced analytical tasks. Liu et al. (2019) extended extremum graphs for the visualization and structural analysis of high-dimensional scalar functions, demonstrating remarkable scalability and analytical performance. Lukaszczuk et al. (2023) further leveraged extremum graphs as an intermediate representation to propose the *ExTreeM* method for efficiently computing enhanced merge trees. By constructing merge trees on extremum graphs rather than performing direct hierarchical merging, this approach dramatically reduces computational complexity and achieves orders-of-magnitude acceleration on multiple public datasets, offering a scalable new paradigm for large-scale data segmentation and analysis. More recently, Das et al. (2024) extended extremum graph analysis to dynamic domains, enabling the capture of temporal topological events such as the creation, annihilation, splitting, and merging of extrema. Their optimization-based framework demonstrates strong performance on complex unsteady datasets, including viscous jets and three-dimensional von Kármán vortex streets.

2.4. Motivation

The current mainstream methods for constructing extremal graphs still exhibit two major limitations:

- (1) **Strong dependency on structured grids.** Most existing algorithms for extremum graph construction heavily rely on the topological regularity of structured grids, such as fixed adjacency patterns and regular connection schemes. In these approaches, both saddle identification and path construction are formulated under the assumption of a regular grid topology. However, in practical applications—such as computational fluid dynamics simulations, medical imaging, and geological modeling—scientific data are often represented as unstructured or hybrid grids, making these structured-grid-based methods difficult to apply directly.

To circumvent this limitation, existing studies typically resample or interpolate unstructured data into structured formats. Nevertheless, such preprocessing not only introduces substantial computational overhead but also accumulates numerical errors, thereby degrading the accuracy and stability of the resulting extremum graph. Consequently, there is an urgent need for a unified construction method that operates directly on arbitrary grid structures without assuming regularity, thus improving both generality and topological fidelity.

- (2) **High computational cost of point-wise saddle detection.** Traditional approaches generally perform a global traversal of all grid points to determine whether each point is a saddle. This point-by-point topological analysis becomes prohibitively expensive when processing large-scale grids—particularly unstructured ones—where the irregular adjacency relationships prevent efficient acceleration. Existing strategies mostly rely on brute-force search or local topological classification, leading to a time complexity that scales linearly with the number of grid points, which is unsuitable for real-time or large-scale data analysis.

To address these challenges, this paper introduces a fast extremum graph construction framework applicable to arbitrary grid structures. The proposed method supports large-scale datasets of arbitrary dimensionality, exhibits strong parallel scalability, and achieves a 3–6× performance improvement over existing techniques while preserving strict topological correctness. Memory consumption is reported in the experimental section; however, it is neither a primary design objective nor a criterion for method comparison. To promote reproducibility and facilitate further research, we have made the source code and example datasets of AnyEG publicly available at <https://github.com/BuptCIAGroup/AnyEG>.

3. Algorithm

The AnyEG algorithm proposed in this work is designed to efficiently identify local extrema and the saddles connecting distinct extremum regions within a scalar field, thereby enabling the construction of an extremum graph that captures the global topological structure.

3.1. The overview of AnyEG

First, the overall workflow of the AnyEG algorithm is presented in Algorithm 1, while the key steps are illustrated in Fig. 2. The input is an arbitrary-dimensional (structured or unstructured) scalar field $S = (V, Neighbors, f)$, where V denotes the set of points, $Neighbors$ encodes the adjacency relations, and $f(v)$ is the scalar value at point v , i.e., the discrete Morse function. The objective of the algorithm is to extract extrema and saddles and to establish flow information directing each point to its corresponding extremum, thereby constructing a structurally concise and semantically clear extremum graph.

Taking the construction of a maximum graph as an example, the procedure involves the following critical steps:

- **Step 0: Data Preprocessing.** The function f must be injective over the input points. This can be enforced using a variant of the Simulation of Simplicity (Edelsbrunner and Mücke, 1990), which preserves the relative order of the original scalar values while ensuring uniqueness across the domain, thereby guaranteeing the correctness of the constructed extremum graph.
- **Step 1: Extrema Detection and Initial Flow Direction.** Identify all local maxima to form the set of extrema $\mathcal{P}_{extrema}$. Simultaneously, assign each point its flow direction along the gradient path, establishing a mapping from points to extrema. As shown in Fig. 2(b), red points denote local maxima, and the arrows indicate initial flow directions. Further details are provided in Section 3.3.
- **Step 2: Path Compression and Control Areas Identification.** Compress the initial flow paths so that each point directly points to its corresponding extremum, resulting in a compact representation and an efficient parent-pointer structure for queries (see Fig. 2(c)). Detailed description is provided in Section 3.4.
- **Step 3: boundary saddle point Detection.** Detect candidate saddles located at the interfaces between different basins of attraction, i.e., transition points connecting two or more extremum basins. Green points in Fig. 2(d) represent the identified boundary saddle points. Detailed explanations are given in Section 3.5.
- **Step 4: Refinement of Saddles.** Using the candidate saddles and Lemma 1, discrete Morse theory is applied for further discrimination. For three-dimensional mesh data, a point whose lower link is non-empty and disconnected is considered to have multiple descent directions, exhibiting saddle characteristics. This criterion effectively eliminates pseudo-saddles, producing a topologically consistent and semantically clear set of saddles $\mathcal{P}_{saddles}$. Purple points in Fig. 2(e) denote the $(n - 1)$ -saddles identified. Further details are provided in Section 3.6.
- **Step 5: Extremum Graph Construction and Filtering.** Based on the identified extrema and refined saddles, an initial extremum graph is constructed. Saddles serve as intermediaries connecting different extrema, forming “saddle-extremum” arcs, as shown in Fig. 2(f). Detailed description is provided in Section 3.7.

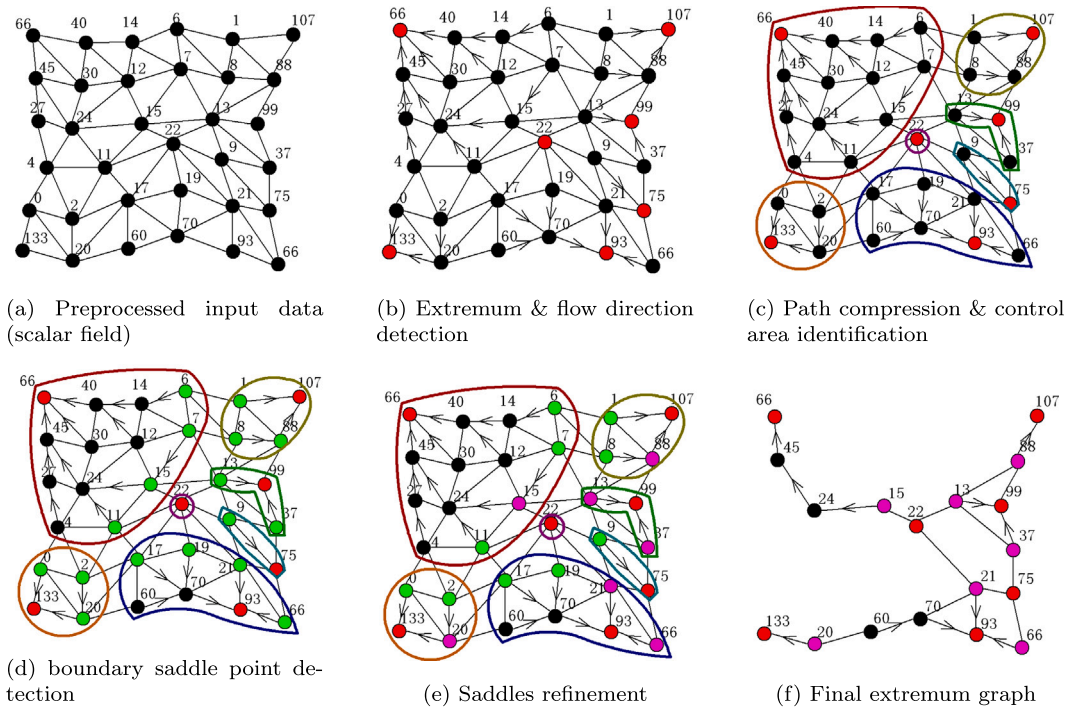


Fig. 2. Overview of the extremum graph construction workflow. Panels (a)–(f) depict the main steps: (a) preprocessed input scalar field, (b) identification of extrema and initial flow directions, (c) path compression and control area determination, (d) boundary saddle point detection, (e) refinement of saddles, and (f) the final extremum graph. Red points denote extrema, green points boundary saddle points, purple points $(n - 1)$ -saddles selected via discrete Morse theory, and black points regular points along the steepest gradient paths. Each closed solid curve represents an extremum control area. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Algorithm 1: The overview of AnyEG.

Input: Scalar field $S = (V, \text{Neighbors}, f)$
Output: set of extrema $\mathcal{P}_{\text{extrema}}$, saddle set $\mathcal{P}_{\text{saddles}}$, point flow information InitialFatherLink.

- 1 $(\mathcal{P}_{\text{extrema}}, \text{FatherLink}) \leftarrow \text{IdentifyExtremumAndInitialFlow}(V, f, \text{Neighbors})$;
- 2 $\text{InitialFatherLink} \leftarrow \text{Copy}(\text{FatherLink})$;
- 3 $\text{ExtremeIndex} \leftarrow \text{CompressPaths}(V, \text{FatherLink})$;
- 4 $\mathcal{P}_{\text{boundary_saddles}} \leftarrow \text{IdentifyBoundarySaddles}(V, f, \text{Neighbors}, \text{ExtremeIndex})$;
- 5 $\mathcal{P}_{\text{saddles}} \leftarrow \text{RefineSaddlesByMorseTheory}(V, f, \text{Neighbors}, \mathcal{P}_{\text{boundary_saddles}})$;
- 6 **return** $(\mathcal{P}_{\text{extrema}}, \mathcal{P}_{\text{saddles}}, \text{InitialFatherLink})$;

The algorithm is described in detail below.

3.2. Data preprocessing and initialization

The construction of the extremum graph is highly dependent on the precise characterization of input data and the rigorous definition of its topological structure. This paper studies scalar fields on arbitrary grids in arbitrary dimensions, assuming that each point corresponds to a unique scalar value. Therefore, during the data preprocessing stage, appropriate processing strategies (Edelsbrunner and Mücke, 1990) must be employed to achieve the uniqueness of scalar values across the entire domain without altering the original relative order of the scalar values. It is noteworthy that although this operation guarantees the uniqueness of scalar values at points, thereby ensuring the correct construction of the extremum graph, it incurs certain overheads: for a grid containing N points, it may introduce a time complexity of $O(N \log N)$ (due to sorting and perturbation operations) and an additional memory overhead of $O(N)$. Moreover, on large-scale grids, this step imposes certain limitations on GPU parallelization and may cause minor effects on numerical precision; these aspects will not be further elaborated in the subsequent analysis.

In this algorithm, the neighbor function $\text{Neighbors}(v)$ is used to store all adjacent nodes of a point v . When implemented on the GPU, careful consideration must be given to how adjacency information is stored and queried to enable efficient parallel access. For structured grids, since nodes are arranged on a regular 3D lattice, the neighbors of each node can be directly computed using fixed offsets and node indices. Therefore, no additional memory allocation is required to store neighbor information, which significantly reduces GPU memory consumption and allows for fast access within GPU threads.

For unstructured grids, however, the number of neighbors and connectivity relationships are irregular, requiring explicit construction of adjacency information during the data loading stage. Specifically, a one-dimensional array NeighborNums stores the number of neighbors for each node i , while a two-dimensional array Neighbors stores the indices of each node's neighbors, where $\text{Neighbors}[i][j]$ represents the index of the j -th neighbor of node i . This storage structure allows the data to reside in contiguous memory blocks on the GPU, facilitating fast access to any node's neighbors through thread indices while maintaining parallel computation efficiency. In this way, whether for structured or unstructured grids, the algorithm can efficiently perform neighbor queries and subsequent extremum or saddle point computations on the GPU.

For the sake of conciseness, this paper will, by default, take maximal graphs as examples for detailed discussion.

3.3. Extremum identification and initial flow direction determination

This stage establishes the foundation for constructing the extremum graph, with the primary objective being the precise identification of maxima and assigning reasonable flow directions to the remaining points to support subsequent analysis. To this end, the “father pointer” (FatherLink) data structure is introduced: it is an array of point pointers used to record the flow direction of each point.

The determination of extrema relies on local comparisons of discrete scalar fields on a grid structure: when the scalar value $f(v)$ of a point v is greater than the scalar values of all its neighbors, i.e., $f(v) > f(n), \forall n \in \text{Neighbors}(v)$, it is determined to be a maximum. For maxima, the parent pointer should directly point to itself, indicating that the flow terminates here (see Algorithm 2, line 16). The definition of a minimum is similar, but the comparison operator is inverted. This determination process essentially involves a discretization of the local gradient direction, using the extrema in the topological neighborhood as the target points for the flow (see Algorithm 2, lines 6-11).

For non-extrema, an initial “parent pointer” must be assigned to its neighbor with the highest scalar value in its neighborhood, thereby simulating gradient flow in discrete space (see Algorithm 2 lines 12–14). This “steepest ascent” flow assignment strategy essentially corresponds to a discrete simulation of continuous gradient flow theory, ensuring that each point ultimately converges to a maximum.

It is worth emphasizing that the entire process is executed in parallel on a point-by-point basis. Each point only needs to be compared locally with points in its neighborhood, without the need for global synchronization, which greatly improves the efficiency of large-scale calculations on parallel architectures such as GPUs. The pseudocode implementation is shown in Algorithm 2.

Algorithm 2: Identify extremum and initial flow.

Input: Scalar field $S = (V, \text{Neighbors}, f)$
Output: $\mathcal{P}_{\text{extrema}}$, FatherLink

```

1 Initialize  $\mathcal{P}_{\text{extrema}} \leftarrow \emptyset$ ;
2 Initialize FatherLink[ $v$ ]  $\leftarrow$  null for all  $v \in V$ ;
3 for each  $v \in V$  in parallel do
4    $is\_maximum \leftarrow \text{true}$ ;
5    $max\_neighbor \leftarrow \text{null}$ ,  $max\_value \leftarrow -\infty$ ;
6   foreach  $n \in \text{Neighbors}(v)$  do
7     if  $f(n) > f(v)$  then
8        $is\_maximum \leftarrow \text{false}$ ;
9     if  $f(n) > max\_value$  then
10       $max\_neighbor \leftarrow n$ ;
11       $max\_value \leftarrow f(n)$ ;
12   if  $is\_maximum$  then
13     FatherLink[ $v$ ]  $\leftarrow v$ ;
14     Add  $v$  to  $\mathcal{P}_{\text{extrema}}$ ;
15   else
16     FatherLink[ $v$ ]  $\leftarrow max\_neighbor$ 
17 return ( $\mathcal{P}_{\text{extrema}}$ , FatherLink);

```

Algorithm 3: Compress paths.

Input: Scalar field $S = (V, \text{Neighbors}, f)$, FatherLink.
Output: updated FatherLink (Recorded as ExtremeIndex).

```

1  $changed \leftarrow \text{true}$ ;
2 while  $changed$  do
3    $changed \leftarrow \text{false}$ ;
4   for each  $v \in V$  in parallel do
5     if FatherLink[ $v$ ]  $\neq$  FatherLink[FatherLink[ $v$ ]] then
6       FatherLink[ $v$ ]  $\leftarrow$  FatherLink[FatherLink[ $v$ ]];
7        $changed \leftarrow \text{true}$ ;
8 return FatherLink

```

3.4. Path compression and control areas identification

The path compression idea originates from the union-find data structure, whose theoretical basis lies in using pointer jumps to eliminate redundant intermediate points. Specifically, for each point v , repeatedly set its parent pointer to point to the parent point of its parent point, i.e., execute $\text{FatherLink}(v) \leftarrow \text{FatherLink}(\text{FatherLink}(v))$ (corresponding to line 6 of Algorithm 3). This process continues until $\text{FatherLink}(v)$ directly points to an extremum (i.e., the parent pointer no longer changes).

In the actual implementation, parallel pointer jumps are used on a per-point basis, synchronously updating the parent pointers of all points in each round and iterating continuously until convergence (see lines 2–7 of Algorithm 3 for details). This strategy fully leverages the parallel computing capabilities of modern GPUs, enabling efficient execution of the path compression process even on large-scale data.

Ultimately, the FatherLink array after path compression serves as the unique identifier for the extremum control area to which each point belongs, denoted as $\text{ExtremeIndex}(v)$. At this point, $\text{ExtremeIndex}(v)$ must be the index of an extremum, indicating that v belongs to the control area represented by that extremum.

3.5. Detection of boundary saddle points

First, we introduce several related concepts. Before providing formal definitions, an intuitive description is given to illustrate the basic notions of extremum control regions and boundary saddle points. In brief, each point in the scalar field “climbs” along its steepest ascent path until reaching a peak (extremum point). All points that eventually flow to the same peak collectively form a *control region*. *boundary saddle points* appear at the interfaces between adjacent control regions, lying on the boundary of two flow domains, as illustrated in Fig. 2(c). Formal definitions of extremum control regions and boundary saddle points are provided below.

Extremum Control Area: Each extremum ep naturally corresponds to a control area, which is composed of all points that ultimately converge to ep along the steepest ascending path:

$$CA(ep) = \{p \in V(M) \mid \text{steepest ascent path from } p \text{ reaches } ep\} \quad (7)$$

Since the steepest path for each point is unique, each point belongs to only one extremum region, and the control area does not intersect with each other and jointly cover the entire point set:

$$\bigcup_{ep} CA(ep) = V(M), \quad CA(ep_i) \cap CA(ep_j) = \emptyset \quad (i \neq j) \quad (8)$$

In the implementation, an array $eCA[i]$ is introduced to represent the extremum to which point i belongs. At initialization, $eCA[i]$ is assigned to the steepest ascending neighbor of point i . In particular, if point i is an extremum, then $eCA[i] = i$. This array can be iteratively updated using a path compression strategy:

$$\text{while } eCA[i] \neq i, \quad eCA[i] \leftarrow eCA[eCA[i]] \quad (9)$$

This process ultimately directs each point to its corresponding extremum, thereby achieving regional division.

Boundary Saddle Point: Let the control area of the extremum u be $CA(u)$. If there exists a point p in $CA(u)$ such that a neighbor of p , p_i , belongs to the control area $CA(v)$ of another extremum v , and $f(p) < f(p_i)$, i.e., p can “flow” to p_i , then p is called a *boundary saddle point*.

For any saddle s , since it must be reachable from at least two extrema, and according to the definition of extremum regions, each point belongs to only one extremum region, there must exist a neighbor s_i such that $eCA[s] \neq eCA[s_i]$, and s can “flow” to s_i . This leads to the following lemma:

Lemma 1. *Any saddle must be a boundary saddle point.*

Based on [Lemma 1](#), we only need to first identify all boundary saddle points and refine them using discrete Morse theory to obtain the $(n-1)$ -saddles (or 1-saddles) required to construct the extremum graph.

It should be noted that the identification of a boundary saddle point depends on the neighbor information of the node. Nodes located on the grid boundaries are inapplicable to this general criterion due to the absence of partial neighboring nodes. Therefore, nodes on the six boundary faces of the grid are processed separately using discrete Morse theory to determine their saddle properties.

Saddles play a pivotal role in connecting different extremum regions within the extremum graph structure, making them crucial for understanding and partitioning the topological structure of complex scalar fields. According to [Lemma 1](#), all saddles are boundary saddle points. If the boundary saddle point can be determined quickly, the overhead of saddle determination is effectively alleviated.

Theoretically, the determination of the boundary saddle point is based on the following intuitive idea: if a point v has at least one neighbor n_i that satisfies the following two conditions:

1. n_i belongs to an extremum control area different from v , i.e., $ExtremeIndex(v) \neq ExtremeIndex(n_i)$;
2. there is a “flow” relationship along the gradient direction between v and n_i , i.e., in the maximum graph, it means $f(v) < f(n_i)$, and in the minimum graph, it means $f(v) > f(n_i)$.

In this case, point v is located at the boundary between two extremum control areas, and its local characteristics reflect the topological properties of the boundary region, potentially forming a saddle (corresponding to lines 4–7 in [Algorithm 4](#)).

The detailed implementation is shown in [Algorithm 4](#). First, initialize the boundary saddle point set $\mathcal{P}_{boundary_saddles}$ to be empty. Then, the following process is executed in parallel for each point v : all neighbors n are examined in turn. If a neighbor n is found that satisfies $f(v) < f(n)$ and has a different extremum region index, then v is immediately determined to be a candidate saddle, and the

Algorithm 4: Identify boundary Saddles.

Input: Scalar field $S = (V, \text{Neighbors}, f)$, extremum region index $ExtremeIndex$.

Output: boundary saddle point set $\mathcal{P}_{boundary_saddles}$.

```

1 Initialize  $\mathcal{P}_{boundary\_saddles} \leftarrow \emptyset$ ;
2 for each  $v \in V$  in parallel do
3    $is\_boundary \leftarrow \text{false}$ ;
4   foreach  $n \in \text{Neighbors}(v)$  do
5     if  $f(v) < f(n)$  and  $ExtremeIndex[v] \neq ExtremeIndex[n]$  then
6        $is\_boundary \leftarrow \text{true}$ ;
7       break;
8   if  $is\_boundary$  then
9     Add  $v$  to  $\mathcal{P}_{boundary\_saddles}$ ;
10 return  $\mathcal{P}_{boundary\_saddles}$ ;

```

examination of other neighbors is terminated. All points that satisfy the conditions are summarized into the boundary saddle point set, which serves as the basis for subsequent steps.

3.6. Saddles refinement based on Morse theory

The essential characteristic of a saddle lies in the presence of more than one direction of “outflow” or “inflow” paths within its neighborhood, reflecting connections to multiple extremum regions. Among the boundary saddle points initially identified, there are often some pseudo-saddles caused by discrete errors or noise. To eliminate these pseudo-saddles, this section refines them further based on discrete Morse theory by analyzing the local topological structure of the boundary saddle point neighborhood.

It is worth emphasizing that the entire process relies solely on point neighborhood information and scalar values, and is independent of the regularity of the grid, thereby making it applicable to any type of discrete space. Additionally, since the previous stage has efficiently filtered out a large number of non-saddles, the current candidate set is of limited size, allowing this step to perform parallel discrimination on a smaller set, effectively reducing the overall computational cost.

The core idea of the algorithm is to utilize the concept of “connection” in Morse theory to examine the upper connection $L_{upper}(s)$ of each boundary saddle point s , i.e., the set of points in all neighborhoods whose scalar values are strictly greater than $f(s)$. According to Morse theory, if removing s results in two or more disconnected connected components among these neighbors, it indicates that s is precisely located at the “boundary” of multiple high-value regions and is a saddle. Conversely, if the connected components of the high-value neighbors do not exceed one, the point s does not actually serve as a bridge connecting different regions and should be excluded because it is not a saddle.

In terms of specific implementation, the algorithm uses boundary saddle points as the parallel granularity. For each s , it first traverses all its neighbors and forms an upper connected set $L_{upper}(s)$ consisting of neighbors with scalar values higher than $f(s)$. Next, on the induced subgraph of $L_{lower}(s)$, the Disjoint Set Union (DSU) algorithm is used to determine the number of connected components: for each pair of neighbors $u, w \in L_{upper}$, if they are adjacent in the original graph, the sets are merged. Finally, count the number of independent sets. If it is greater than one, retain s as the refined saddle; otherwise, discard it. This process is detailed in Algorithm 5, which is naturally suited for parallel processing by each boundary saddle point.

3.7. Extremum graph construction and filtering

First, based on the flow direction information and affiliation information obtained in the previous stage (which extremum each point ultimately belongs to), the neighborhood structure of each saddle is examined in parallel across the entire domain. By traversing

Algorithm 5: Refine Saddles by Morse theory.

Input: Scalar field $S = (V, \text{Neighbors}, f), \mathcal{P}_{\text{boundary_saddles}}$
Output: Refined saddle set $\mathcal{P}_{\text{saddles}}$

```

1 Initialize  $\mathcal{P}_{\text{saddles}} \leftarrow \emptyset$ ;
2 for each  $s \in \mathcal{P}_{\text{boundary\_saddles}}$  in parallel do
3    $L_{lower} \leftarrow \emptyset$  foreach  $n \in \text{Neighbors}(s)$  do
4     if  $f(n) < f(s)$  then
5       Add  $n$  to  $L_{lower}$ ;
6   if  $\text{size}(L_{lower}) \leq 1$  then
7     continue
8   Initialize Disjoint Set Union (DSU) structure for elements in  $L_{lower}$ ;
9   foreach  $u \in L_{lower}$  do
10    DSU.MakeSet( $u$ );
11   foreach  $u \in L_{lower}$  do
12     foreach  $w \in L_{lower}, w \neq u$  do
13       if  $w \in \text{Neighbors}(u)$  then
14         DSU.Union( $u, w$ );
15    $\text{num\_components} \leftarrow 0$ ;
16   foreach  $u \in L_{lower}$  do
17     if DSU.Find( $u$ ) ==  $u$  then
18        $\text{num\_components} \leftarrow \text{num\_components} + 1$ ;
19   if  $\text{num\_components} > 1$  then
20     Add  $s$  to  $\mathcal{P}_{\text{saddles}}$ ;
21 return  $\mathcal{P}_{\text{saddles}}$ ;

```

all directly adjacent grid points, the set of extrema that can be reached within the neighborhood is counted. Furthermore, for each pair of extrema connected by a saddle, an edge is recorded along the flow path between the saddle and the extrema, along with the scalar value and other attribute information of the saddle.

However, the initially generated extremum graph often contains a large number of redundant or weak connections. To address this, we first calculate the topological persistence of each “saddle-extremum” arc to measure the significance of the connection in the global structure. By setting a reasonable persistence threshold, we can parallelly remove weak connections that have a minor impact on the overall shape. Additionally, since multiple connection arcs may exist between the same pair of extrema via different saddles, an arc bundling strategy is implemented, retaining only the connection corresponding to the saddle with the largest scalar value. This filtering operation not only enhances the sparsity of the extremum graph but also strengthens the expressive capability of the skeleton structure, facilitating subsequent feature analysis and visualization.

Through strategies such as parallel traversal centered on saddles, path tracing, and arc screening, efficient and stable extremum relationship modeling was achieved. The final output is a concise and stable extremum graph, which provides a solid foundation for further applications such as topological analysis and feature extraction.

4. Experiments and discussion

4.1. Experimental environment and methods

All experiments were conducted on a workstation equipped with an Intel(R) Core(TM) i7-11700 @ 2.50GHz processor, which features 8 cores and 16 threads. The machine has 128 GB of memory and an NVIDIA GeForce RTX 3060 graphics card (12 GB of VRAM). The operating system is Ubuntu 18.04.6 LTS, which offers excellent stability and broad software compatibility.

All algorithms in the experiment were implemented in C++ and combined with CUDA 10.2 for GPU-accelerated programming. The compiler used was GCC 7.5.0, which ensures good support for the C++14 standard. All code underwent multiple rounds of optimization and debugging to maximize algorithm efficiency by utilizing GPU parallel computing resources. At the same time, to ensure the fairness of the experiment, all code was run in the same hardware and software environment.

To verify the performance and applicability of the proposed extremum graph construction algorithm for the system, two types of experiments were designed, targeting regular grid and irregular grid data, respectively.

Regular grid performance testing and effect verification: Standard 3D regular grid data is selected as input, with a focus on evaluating the computational efficiency of the algorithm. The shared-memory extremum graph algorithm (i.e., Tachyon) (Ande et al., 2023) is used for performance comparison. By comparing the runtime and calculating the speedup between the two algorithms, the objective effectiveness of AnyEG in regular grid scenarios is assessed. In the experiments, multiple independent tests were conducted on regular grids of varying scales and complexities to ensure the stability and representativeness of the results.

Non-regular grid performance testing and effect verification: To evaluate the applicability of AnyEG on more complex and realistic irregular grid data, various types of unstructured grid samples were used as input. The resulting extremum graphs were stored in VTK file format, enabling intuitive visualization and verification through general-purpose visualization tools.

In the experiment, the performance indicators mainly included the running time of each module, the total running time, and the speedup. In terms of effectiveness, visualization images were mainly used to assist in displaying the identification of extrema and the topological consistency of connection relationships. All experiments were performed multiple times and the results were averaged to reduce random errors.

4.2. Results and discussion

4.2.1. Experiments on structured grids

The first group of experiments is based on Tachyon (Ande et al., 2023). A total of 29 structured datasets were selected, covering different resolutions and topological complexities. We divided the computation process of the proposed method into four critical steps, statistically analyzed the runtime of each step and the total runtime, and compared them with the Tachyon (Ande et al., 2023) to calculate the speedup.

The experimental results are presented in Tables 1 and 2, and can be further visualized in Figs. 4 and 5, which illustrate the stepwise runtime distribution, VRAM usage with respect to the number of grid points, and the runtime speedup comparisons, respectively. AnyEG demonstrates significant acceleration effects on most datasets, with overall runtime efficiency superior to the Tachyon method. For some large-scale datasets, due to the limited memory capacity of the RTX 3060 graphics card, we removed some auxiliary arrays to conserve resources, resulting in some information needing to be recalculated during runtime. Despite this, the experimental results still demonstrate that our optimizations in algorithm structure design and module implementation achieve good results across different dataset sizes, showcasing strong adaptability and scalability. We will provide a detailed analysis of the algorithm’s time and memory consumption in Sections 4.3 and 4.4.

While ensuring efficiency, we also verified the topological correctness of the output results. By simplifying the extremum graph results of three typical data sets according to the topological persistence of each saddle extremum pair, we eliminated saddle extremum pairs with too little topological persistence to avoid confusion. As shown in Fig. 3, we can observe that the extremum graph we generated is reasonable and consistent with expectations in terms of critical point locations and gradient path connections, indicating that the algorithm achieves significant performance improvements while maintaining correctness.

Table 1
Runtime across computational steps and VRAM usage across structured grid datasets.

Dataset	Pts	Step 1 (s)	Step 2 (s)	Step 3 (s)	Step 4 (s)	VRAM Usage (GB)
silicium	113,288	0.00028	0.00013	0.00002	0.00052	0.001
fuel	262,144	0.00034	0.00016	0.00004	0.00027	0.002
neghip	262,144	0.00032	0.00018	0.00004	0.00074	0.002
hydrogen_atom	2,097,152	0.00139	0.00128	0.00059	0.00157	0.014
shockwave	2,097,152	0.00129	0.00123	0.00065	0.00092	0.014
lobster	5,461,344	0.00332	0.00365	0.00103	0.01963	0.036
mri_ventricles	8,126,464	0.00525	0.00556	0.00215	0.05931	0.053
engine	8,388,608	0.00522	0.00549	0.00163	0.02805	0.055
statue_leg	10,814,133	0.00651	0.00712	0.00273	0.01887	0.071
boston_teapot	11,665,408	0.00636	0.00751	0.00196	0.01446	0.076
aneurism	16,777,216	0.00873	0.01014	0.00330	0.00800	0.109
bonsai	16,777,216	0.00921	0.01054	0.00335	0.03612	0.109
foot	16,777,216	0.00878	0.01102	0.00334	0.06458	0.109
skull	16,777,216	0.00911	0.01158	0.00331	0.11383	0.109
mrt_angio	23,855,104	0.01525	0.02576	0.00522	0.19720	0.156
stent	45,613,056	0.02410	0.03945	0.00936	0.20034	0.297
pancreas	62,914,560	0.03187	0.04898	0.01128	0.41634	0.410
backpack	97,779,712	0.05097	0.09399	0.01817	0.38192	0.637
present	106,992,288	0.05726	0.11481	0.02101	0.47949	0.698
prone	121,372,672	0.06461	0.12648	0.02186	0.82040	0.791
christmas_tree	130,809,856	0.06743	0.14380	0.02681	0.78792	0.853
magnetic_reconnection	134,217,728	0.07093	0.16020	0.02432	1.14345	0.875
zeiss	314,432,000	0.15620	0.30795	0.05169	0.43960	2.050
marmoset_neurons	329,252,864	0.17366	0.40399	0.05523	1.65937	2.146
pawpawsaurus	673,328,384	0.35940	0.82924	0.11445	5.40544	4.390
spathorhynchus	786,432,000	0.40290	0.81145	0.13344	4.39486	5.127
kingsnake	833,617,920	0.42689	0.90910	0.14109	4.02070	5.435
chameleon	1,132,462,080	0.58195	1.22500	0.18924	6.20437	7.383
beechnut	1,621,098,496	1.82800	2.22062	0.28295	7.56461	10.568

Note: Pts represents the total number of grid points ($Pts = d_1 \times d_2 \times d_3$ for 3D structured grids). Each dataset was tested three times, and the step timings from the best run are reported. VRAM usage depends only on Pts and is therefore deterministic, requiring no repeated measurements.

Table 2
Comparison of runtime and speedup between AnyEG and Tachyon on different structured datasets.

Dataset	AnyEG(95%CI) (s)	Tachyon(95%CI) (s)	Speedup(95%CI)
silicium	0.00099±0.00004(0.00089,0.00109)	0.00331±0.00004(0.00322,0.00339)	3.33±0.14(2.99,3.68)
fuel	0.00086±0.00005(0.00073,0.00100)	0.00421±0.00014(0.00386,0.00455)	4.87±0.35(4.00,5.73)
neghip	0.00129±0.00001(0.00127,0.00130)	0.00556±0.00029(0.00484,0.00628)	4.32±0.23(3.76,4.89)
hydrogen_atom	0.00498±0.00025(0.00436,0.00561)	0.02471±0.00032(0.02391,0.02551)	4.96±0.26(4.32,5.60)
shockwave	0.00441±0.00047(0.00323,0.00558)	0.02137±0.00029(0.02064,0.02210)	4.85±0.52(3.55,6.15)
lobster	0.02863±0.00122(0.02560,0.03165)	0.12493±0.00021(0.12442,0.12544)	4.36±0.19(3.90,4.83)
mri_ventricles	0.07266±0.00037(0.07174,0.07358)	0.36265±0.00642(0.34670,0.37861)	4.99±0.09(4.76,5.22)
engine	0.04102±0.00055(0.03964,0.04239)	0.18836±0.00045(0.18723,0.18949)	4.59±0.06(4.44,4.75)
statue_leg	0.03535±0.00015(0.03499,0.03571)	0.19493±0.00132(0.19166,0.19820)	5.51±0.04(5.41,5.62)
boston_teapot	0.03047±0.00015(0.03008,0.03085)	0.13051±0.00043(0.12943,0.13158)	4.28±0.03(4.22,4.35)
aneurism	0.03088±0.00119(0.02793,0.03383)	0.15234±0.00059(0.15087,0.15382)	4.93±0.19(4.46,5.41)
bonsai	0.05992±0.00068(0.05824,0.06160)	0.21889±0.00079(0.21693,0.22086)	3.65±0.04(3.55,3.76)
foot	0.08848±0.00080(0.08650,0.09045)	0.29101±0.00342(0.28251,0.29950)	3.29±0.05(3.17,3.41)
skull	0.13802±0.00025(0.13740,0.13863)	0.54147±0.00477(0.52962,0.55332)	3.92±0.04(3.84,4.01)
mrt_angio	0.24652±0.00422(0.23604,0.25700)	1.03623±0.01255(1.00506,1.06741)	4.20±0.09(3.98,4.42)
stent	0.27760±0.00388(0.26798,0.28723)	0.98143±0.00439(0.97052,0.99233)	3.54±0.05(3.41,3.66)
pancreas	0.51043±0.00218(0.50502,0.51585)	1.91553±0.00779(1.89618,1.93488)	3.75±0.02(3.70,3.81)
backpack	0.54585±0.00085(0.54374,0.54797)	2.25079±0.04359(2.14252,2.35907)	4.12±0.08(3.92,4.32)
present	0.67616±0.00312(0.66842,0.68390)	3.66841±0.03889(3.57179,3.76502)	5.43±0.06(5.27,5.58)
prone	1.03651±0.00443(1.02552,1.04751)	3.60848±0.01624(3.56814,3.64882)	3.48±0.02(3.43,3.54)
christmas_tree	1.03004±0.00359(1.02112,1.03896)	4.84924±0.00868(4.82768,4.87079)	4.71±0.02(4.66,4.75)
magnetic_reconnection	1.40535±0.00565(1.39131,1.41939)	6.46169±0.11103(6.18589,6.73749)	4.60±0.08(4.40,4.80)
zeiss	0.97257±0.01492(0.93550,1.00964)	3.90929±0.01475(3.87264,3.94594)	4.02±0.06(3.86,4.18)
marmoset_neurons	2.30757±0.01995(2.25800,2.35714)	13.84212±0.18193(13.39017,14.29406)	6.00±0.09(5.76,6.23)
pawpawsaurus	6.72047±0.02009(6.67057,6.77037)	22.75877±0.40270(21.75841,23.75914)	3.39±0.06(3.24,3.54)
spathorhynchus	5.74846±0.00535(5.73518,5.76175)	18.28493±0.08245(18.08011,18.48975)	3.18±0.01(3.14,3.22)
kingsnake	5.51767±0.01870(5.47122,5.56412)	21.11910±0.31820(20.32864,21.90956)	3.83±0.06(3.68,3.97)
chameleon	8.24827±0.05762(8.10512,8.39141)	24.89651±0.08626(24.68224,25.11078)	3.02±0.02(2.96,3.08)
beechnut	15.38200±3.55792(6.54364,24.22036)	42.91557±0.58894(41.45257,44.37856)	2.79±0.65(1.18,4.40)

Note: Tachyon refers to the parallel extremum graph construction method proposed by Ande et al. [Ande et al. \(2023\)](#). The runtime and speedup of AnyEG and Tachyon are reported as mean ± standard deviation (95% confidence interval), averaged over three independent runs.

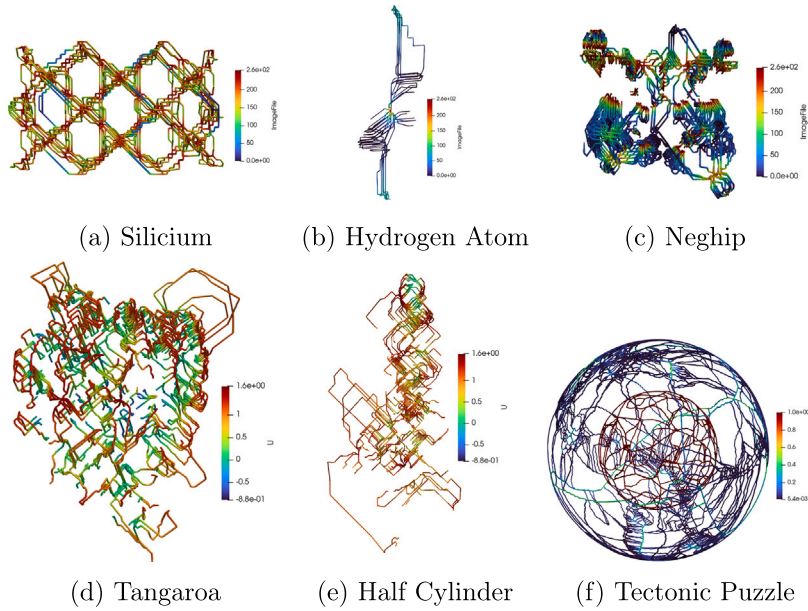


Fig. 3. Simplified extremum graphs for various datasets. points (extrema and saddles) are not shown to reduce clutter, and scalar fields are mapped to colors shown in the legend. Panels (a)–(c) show structured grid datasets, while (d)–(f) show unstructured grid datasets. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

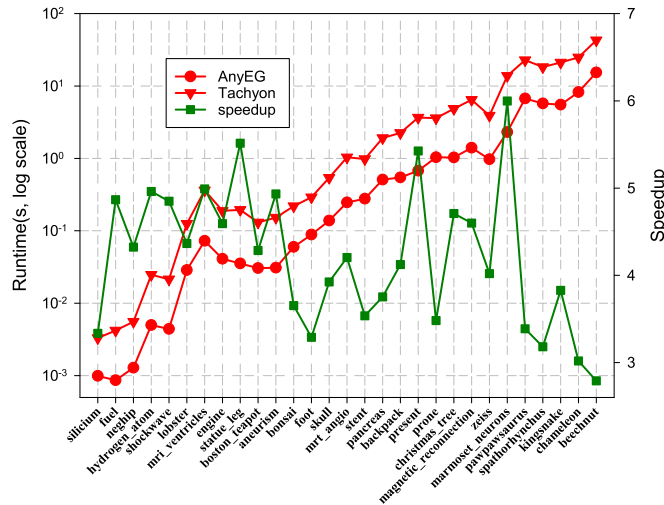


Fig. 4. Comparison of runtime and speedup between AnyEG and Tachyon on structured grid datasets. For better readability, the runtime data is plotted on a logarithmic scale; the left vertical axis represents runtime in log scale, while the right vertical axis shows the speedup.

4.2.2. Experiments on unstructured grids

To further verify the generality and robustness of the method, we conducted tests on three sets of unstructured grid datasets. Due to the irregular cell connection structure and complex topology of this type of data, higher requirements are placed on the construction of extremum graphs.

The experimental results demonstrate that the proposed method can not only stably process all unstructured datasets and successfully generate the corresponding extremum graph structures, but also exhibits good performance in terms of efficiency. We also calculated the runtime of the four critical modules and the overall computation time, with the specific results shown in Table 3. Although unstructured grids present certain challenges in data access and adjacency relationship construction, all modules ran stably with efficiency within acceptable ranges, further validating the scalability of the method.

In addition, to verify the topological correctness of the results, we performed the same simplification operations on the extremum graphs of three typical datasets as we did on the structured grid dataset, as shown in Fig. 3. The distribution of maxima and minima in the figure is reasonable, the gradient paths are clear, and the structural consistency is good, consistent with the expected topological structure, indicating that the proposed method still has strong adaptability under complex grid conditions.

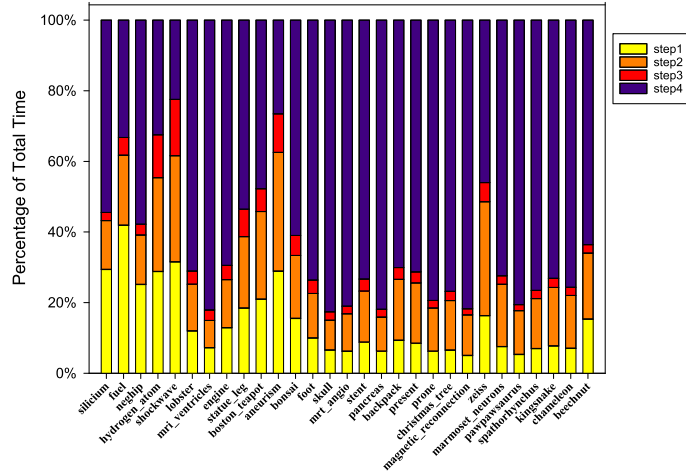


Fig. 5. Analysis of the proportion of runtime consumed by each step in AnyEG across different datasets.

Table 3

Runtime of various unstructured grid datasets for different computational steps.

Dataset	Pts	Step 1 (s)	Step 2 (s)	Step 3 (s)	Step 4 (s)	Total time (s)
tectonicPuzzle	184,112	0.00130	0.00017	0.00015	0.00044	0.00205
tangaroa3d	377,367	0.00617	0.00022	0.00047	0.03861	0.04546
halfcylinder3d	368,851	0.00638	0.00022	0.00067	0.02001	0.02728

Note: The specific usage data are the T scalar values of 640_halfcylinder-U15.00.vtu, tangaroa-U2.00.vtu, and tectonicPuzzle.vtu.

In summary, the results of this group of experiments show that our algorithm not only performs well on structured data, but also efficiently and accurately processes unstructured grids, making it suitable for a wider range of scientific computing and visualization scenarios.

4.3. Time and performance analysis

4.3.1. Time complexity analysis of AnyEG

The core steps of AnyEG are implemented as CUDA kernels for parallel execution on GPUs. The following analysis focuses on their parallel time complexity. Let the total number of points be N and the length of the longest steepest ascent path be k (with $k < N$). Using the Work-Span model, we analyze the parallel time complexity of the core functions of AnyEG. The parallel critical path length (Span) is determined to be $O(\log k)$, while the total work (Work) is $O(N \log k)$, with the overall parallel time complexity dominated by Span.

Under the CUDA parallel computing architecture, Work and Span are defined as follows:

- Work: the total number of operations required to complete the task, corresponding to the serial time complexity and reflecting the overall computational scale of the task;
- Span: the execution time of the longest parallel critical path, determining the theoretical minimum parallel execution time. It serves as a key metric for parallel algorithm efficiency, requiring careful analysis of inter-module dependencies and the longest path duration.

The detailed derivation is as follows:

Step 1: Using the CUDA kernel `judgeExtremePoints_Coords`, the extremum properties of all points are determined in parallel, while the host traverses the `isExtremePoint` array to count the number of extremum points.

- Work analysis: the kernel performs local neighborhood comparisons for all N points (operations linearly proportional to $\Delta \cdot N$, where Δ is the number of neighbors per point; for typical 3D structured grids, $\Delta = 14$), and the host traverses an array of length N , giving $\text{Work} = O(\Delta \cdot N)$;
- Span analysis: point-level computations in the kernel are independent (fully parallelizable), with execution time determined by thread block scheduling constants; host traversal is serial ($O(N)$) but can overlap with subsequent modules, with no dependency constraints. Therefore, $\text{Span} = O(\Delta)$ (constant-level).

Step 2: The `FatherLinks` array is iteratively updated via pointer-doubling to trace the steepest ascent path from each point to its corresponding extremum, with the iteration count determined by the longest path length k .

- Work analysis: each iteration updates pointers of N points in parallel using the `updateFa` kernel (single-iteration cost $O(N)$); with doubling steps from 1 to 2^m until $2^m \geq k$, the number of iterations is $m = O(\log k)$. Hence, $\text{Work} = O(N \log k)$;
- Span analysis: the iterative process has strict dependencies—iteration i depends on iteration $i - 1$ (the doubling step relies on previous data), forming a serial chain. Therefore, $\text{Span} = O(\log k)$.

Step 3: Using the CUDA kernel `judgeBoundarySaddlePoints_Coords`, boundary saddle points are determined in parallel. The operation logic is identical to Step 1, with no cross-point dependencies. Thus, $\text{Work} = O(\Delta \cdot N)$, $\text{Span} = O(\Delta)$ (constant-level).

Step 4: The function `launchJudgeSaddle_Coords` calls the CUDA kernel `judgeCandidateSaddlePoints_ByMorse_Coords` to determine saddle points for each point in parallel. The kernel independently performs the following operations: converting point indices to 3D coordinates, collecting neighbors with higher scalar values, initializing union-find structures, merging connected high-value neighbors, and counting connected components. For each point, neighbor traversal and union-find operations are of constant complexity (Δ is fixed).

- Work analysis: each point incurs $O(\Delta^2)$ operations, for a total of N points. Thus, $\text{Work} = O(\Delta^2 \cdot N)$;
- Span analysis: point computations are independent, so the critical path depends only on the most time-consuming operations of a single point (neighbor pair merging and union-find), giving $\text{Span} = O(\Delta^2)$ (constant-level).

The parallel critical path of the AnyEG core algorithm follows the sequence: Step 2 iterative chain $\rightarrow$ Step 3 boundary saddle point determination $\rightarrow$ Step 4 saddle point refinement. The $\text{Span} = O(\log k)$ of Step 2 dominates the overall parallel execution time, whereas the $O(\Delta^2)$ spans of the subsequent steps have negligible impact on the asymptotic complexity.

In summary, based on the Work-Span model and the longest steepest ascent path length k ($k < N$), the time complexity of AnyEG core functions is:

- Total work (Work): $O(N \log k)$, reflecting the overall computational scale in serial execution;
- Parallel critical path (Span, i.e., parallel time complexity): $O(\log k)$, determining the theoretical minimum parallel execution time.

These results indicate that the AnyEG core functions exhibit high parallel scalability on large-scale datasets, with parallel execution time growing logarithmically with k , significantly outperforming traditional serial algorithms' linear growth, making it suitable for topological analysis of large point datasets.

4.3.2. Actual runtime and speedup results of AnyEG

Theoretical complexity reflects only the asymptotic growth of an algorithm, whereas actual performance must be evaluated based on runtime across different datasets. Table 2 and Fig. 4 report the runtime and speedup of AnyEG compared to Tachyon on various structured datasets. Overall, AnyEG consistently outperforms Tachyon, achieving an average speedup of approximately 4.

On small-scale datasets (e.g., *silicium*, *fuel*, *neghip*), AnyEG reaches millisecond-level runtimes with speedups of 3–5, demonstrating the benefits of a low-overhead, highly optimized data processing pipeline. On medium-scale datasets (e.g., *mri_ventricles*, *engine*, *statue_leg*), the speedup remains around 4–5. For large-scale datasets (e.g., *marmoset_neurons*, *pawpawsaurus*, *chameleon*, *beechnut*), despite the significantly increased data volume, AnyEG still achieves speedups of 3–6, indicating good scalability and robustness for processing large-scale grid data. Some extremely large datasets (e.g., *beechnut*) exhibit larger confidence intervals due to fluctuations from complex data distributions, but the overall trend clearly highlights AnyEG's performance advantage. A detailed step-wise analysis is presented below:

- **Step 1: Extremum Identification and Initial Flow Determination:** The main computational cost arises from local neighborhood comparisons and parent pointer initialization. As shown in Table 1, this step requires only milliseconds on small datasets, with a maximum of approximately 0.58 s. While runtime scales linearly with data size, it remains significantly lower than that of Step 4.
- **Step 2: Path Compression and Control Region Identification:** The primary cost comes from multiple iterations of pointer-jumping convergence. Table 1 indicates that for most datasets, this step maintains millisecond-level runtime, and even for datasets exceeding one billion points, it does not exceed 1.23 s, demonstrating excellent parallel efficiency.
- **Step 3: boundary saddle point Detection:** The main cost involves boundary index comparisons and neighborhood access. According to Table 1, even for the largest dataset with 1.6 billion points, runtime remains below 0.3 s, showing minimal sensitivity to data size and benefiting from efficient parallelization.
- **Step 4: Saddle Point Refinement:** The primary cost stems from union-find operations and local connectivity checks. As shown in Table 1, runtime increases significantly with data size—from milliseconds for small datasets to over 7 s for 1.6 billion points—making this the dominant factor in overall runtime.
- **Overall Runtime and Speedup:** As illustrated in Fig. 5, total runtime is dominated by Step 4. Experimental results show that on small to medium datasets, AnyEG achieves several- to ten-fold speedup over traditional methods, with an overall average speedup exceeding 5 (see Table 2).

4.4. Memory usage analysis

4.4.1. Space complexity analysis of AnyEG

In this study, the space complexity is evaluated with the total number of points N as the core problem size, focusing on the asymptotic growth of memory usage with respect to N while ignoring constant auxiliary space that is independent of N during algorithm execution.

For the basic data types involved, the storage space per element is constant, i.e., $O(1)$: half (16-bit floating point, corresponding to `Values_fp16`) typically occupies 2 bytes; bool (corresponding to `isExtremePoint` and `isCandidateSaddlePoint`) typically occupies 1 byte (or 4 bytes due to memory alignment, still constant); int (corresponding to `FatherLinks`) usually occupies 4 or 8 bytes. These storage sizes do not change with N .

Moreover, due to GPU memory limitations, several memory-saving strategies were adopted compared to the original implementation. First, data type compression was applied by replacing the original float array `Values` with the half-precision array `Values_fp16`, which reduces scalar storage cost by 50% while maintaining acceptable computational accuracy. For set storage, the original scheme used int arrays `ExtremePoints` and `CandidateSaddlePoints` to store extremum and boundary saddle point sets, marking deleted elements with -1 and performing compaction. Although this reduces parallel scalability, it incurs additional memory overhead. The optimized version instead uses bool arrays `isExtremePoint` and `isCandidateSaddlePoint`, marking each point with a boolean flag, reducing per-element storage from 4 bytes to 1 byte and substantially lowering memory usage. Additionally, the original int array `ExtremeIndices` was renamed to `FatherLinks` to avoid naming ambiguity while maintaining functionality, and the `NeighborNums` and `Neighbors` arrays for unstructured grids were preserved to ensure algorithm correctness. Although these adjustments forgo some parallel efficiency from set compaction and may cause minor warp divergence due to bool array flags, the significant reduction in GPU memory usage ensures feasibility on GPUs such as the RTX 3060 for large-scale data.

Considering the storage characteristics of each variable: `Values_fp16`, an array of N half elements, occupies total space $C_1 \cdot N$, which asymptotically gives space complexity $O(N)$; the bool arrays `isExtremePoint` and `isCandidateSaddlePoint`, each containing N elements, occupy $C_2 \cdot N$ and $C_4 \cdot N$ bytes, respectively, both $O(N)$; the int array `FatherLinks`, with N elements, occupies $C_3 \cdot N$ bytes, also $O(N)$.

According to the additive rule of space complexity, if multiple data structures have the same asymptotic order, the overall space complexity is determined by a single structure of the same order. Therefore, the four variables mentioned above all have a space complexity of $O(N)$, and their combined asymptotic trend remains $O(N)$.

In summary, `Values_fp16`, `isExtremePoint`, `isCandidateSaddlePoint`, and `FatherLinks` each exhibit linear space complexity $O(N)$, and the overall storage scheme formed by these four variables also has $O(N)$ linear space complexity, i.e., memory usage grows linearly with the problem size N .

4.4.2. Actual VRAM usage of AnyEG

The memory optimization effects discussed theoretically need to be quantitatively validated through actual VRAM monitoring. Below, we present the recorded VRAM consumption during experiments to illustrate the real memory usage of AnyEG across different dataset scales, further confirming the validity of the theoretical space complexity analysis.

As shown in Table 2, the actual VRAM usage of AnyEG demonstrates a linear relationship with the number of grid points (Pts), which is consistent with the theoretical space complexity analysis. For small-scale datasets, such as *silicium* (approximately 110,000 points) and *fuel* (approximately 260,000 points), the memory footprint is extremely low, only 0.001–0.002 GB. As the dataset size increases, memory demand grows gradually. For instance, medium-scale datasets *mri_ventricles* (approximately 8.1 million points) and *engine* (approximately 8.4 million points) require 0.053 GB and 0.055 GB of VRAM, respectively, while large-scale datasets *pawpawosaur* (approximately 670 million points) and *chameleon* (approximately 1.13 billion points) consume 4.39 GB and 7.38 GB, clearly reflecting the growth of memory usage with point count.

Furthermore, for ultra-large datasets, such as *beechnut* (approximately 1.62 billion points), VRAM usage can still be controlled within 10.57 GB, demonstrating the excellent memory scalability of AnyEG on high-performance GPUs. These observations complement the step-wise runtime analysis presented in Table 1, indicating that the memory optimization strategies effectively support large-scale structured grid processing while maintaining high parallel computational efficiency.

5. Limitations and scope

The proposed framework is established upon several theoretical and implementation assumptions that define its scope of applicability and potential limitations.

First, the method assumes that the input scalar field is a simple Morse function defined on a D -dimensional simplicial complex. In other words, the scalar data must be preprocessed through Simulation of Simplicity (SoS) to eliminate possible degeneracies. After this step, all critical points become non-degenerate and possess distinct function values, ensuring that the topological structure of the scalar field can be rigorously characterized by Morse theory. However, this assumption excludes non-Morse or degenerate scalar fields, such as cases involving merged critical points or higher-order singularities. Extending the current framework to handle non-simple Morse functions or even more general piecewise-smooth scalar mappings will be an important direction for future research.

Second, the current algorithm does not yet incorporate an out-of-core processing mechanism. It assumes that the entire input mesh and associated scalar field can be fully loaded into GPU memory during computation. Consequently, when the data size exceeds the physical GPU memory capacity, the system becomes incapable of processing it—particularly for large-scale unstructured grids. Future

work may address this limitation by introducing efficient data streaming, hierarchical partitioning, or hybrid CPU–GPU out-of-core computation mechanisms to further improve the scalability and generality of the algorithm.

Overall, the proposed method achieves efficient and stable extremum-graph computation within its assumed conditions, but it is currently best suited for non-degenerate scalar fields that fit within GPU memory. Subsequent research will focus on relaxing these assumptions to enhance the robustness and scalability of the framework.

6. Conclusion

The Extremum Graph serves as an effective intermediate representation for characterizing topological structures in scalar fields, holding significant value in scientific visualization, feature extraction, and structural simplification tasks. However, mainstream Extremum Graph construction methods currently face two critical challenges. First, they heavily rely on the topological regularity of structured grids, making them incompatible with unstructured or hybrid grid data. Second, most approaches adopt a point-by-point saddle detection strategy, which is computationally expensive and struggles to meet the processing demands of large-scale datasets.

Therefore, a novel fast extremum graph construction method is proposed, denoted by *AnyEG*, which is applicable to arbitrary grids, addressing the aforementioned challenges. We conducted experimental validation on multiple structured and unstructured grid datasets. The results demonstrate that our method achieves significant acceleration compared to existing approaches while preserving topological correctness. Specifically, on medium-sized and small-scale datasets, the average speedup ratio exceeds 5 times, with a maximum acceleration of nearly 48 times. For large-scale datasets, even under GPU-accelerated environments with limited memory resources, our method exhibits robust stability and scalability. Furthermore, the constructed extremum graphs maintain consistency with traditional methods in terms of point distribution, gradient path connectivity, and overall structural topology, confirming that the proposed algorithm achieves an optimal balance between computational accuracy and efficiency.

In summary, *AnyEG* presents an efficient and scalable framework for constructing extremum graphs on arbitrary grids, thereby providing a robust foundation for the topological analysis and visualization of scalar fields. Future work will focus on the following directions:

- Integrating graph neural networks with deep learning methods to enhance feature recognition capabilities while maintaining the interpretability of topological structures;
- Optimize the integration process with visualization tools such as TTK to promote the practical application of extremum graphs in engineering and scientific research.

CRedit authorship contribution statement

Quming Li: Writing – original draft, Validation, Software. **Zhengwen Liu:** Writing – original draft, Visualization, Validation, Software. **Zhibin Huang:** Writing – original draft, Validation, Supervision, Investigation, Formal analysis, Conceptualization. **Wantong Li:** Writing – original draft, Visualization. **Zhiqiang Chu:** Visualization, Validation, Software. **Zhitao Dai:** Resources, Investigation, Formal analysis. **Min Yu:** Funding acquisition, Formal analysis, Data curation, Conceptualization. **Fang Deng:** Resources, Methodology, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The source code and example datasets of *AnyEG* are publicly available at <https://github.com/BuptCIAGroup/AnyEG>.

References

- Ande, A., Subhash, V., Natarajan, V., 2023. Tachyon: efficient shared memory parallel computation of extremum graphs. In: Computer Graphics Forum. Wiley Online Library. p. e14784.
- Bremer, P.-T., Weber, G.H., Tierny, J., Pascucci, V., Day, M., Bell, J., 2010. Topological feature extraction and tracking. J. Phys.: Conf. Ser. 78, 012007.
- Bruckner, S., Miksch, S., Pfister, H., 2012. Parallel computation of 3d morse-smale complexes. In: Proceedings of the Eurographics Conference on Visualization (EuroVis). Eurographics Association.
- Correa, C., Lindstrom, P., Bremer, P.-T., 2011. Topological spines: a structure-preserving visual representation of scalar fields. IEEE Trans. Vis. Comput. Graph. 17, 1842–1851.
- Das, D.P., 2023. Optimization in Extremum Graph Computation. Ph.D. thesis. Indian Institute of Science Bangalore.
- Das, S., Sridharamurthy, R., Natarajan, V., 2024. Time-varying extremum graphs. In: Computer Graphics Forum. Wiley Online Library. p. e15162.
- Edelsbrunner, H., Harer, J., 2010. Computational Topology: An Introduction. American Mathematical Soc.
- Edelsbrunner, H., Mücke, E.P., 1990. Simulation of simplicity: a technique to cope with degenerate cases in geometric algorithms. ACM Trans. Graph. 9, 66–104.
- Forman, R., 1998. Morse theory for cell complexes. Adv. Math. 134, 90–145.
- Gyulassy, A., Bremer, P.-T., Hamann, B., Pascucci, V., 2008. A practical approach to morse-smale complex computation: scalability and generality. IEEE Trans. Vis. Comput. Graph. 14, 1619–1626.
- Gyulassy, A., Natarajan, V., Pascucci, V., Bremer, P.-T., Hamann, B., 2007. Topology-based simplification for feature extraction from 3d scalar fields. In: IEEE Visualization. IEEE, pp. 535–542.
- Gyulassy, A., Pascucci, V., Peterka, T., Ross, R., 2012. The parallel computation of morse-smale complexes. In: 2012 IEEE 26th International Parallel and Distributed Processing Symposium. IEEE, pp. 484–495.

- Liu, S., Cui, W., Wu, Y., Liu, M., 2017. Visualizing high-dimensional data: advances in the past decade. *IEEE Trans. Vis. Comput. Graph.* 23, 1249–1268.
- Liu, S., Wang, D., Maljovec, D., Anirudh, R., Thiagarajan, J.J., Jacobs, S.A., Van Essen, B.C., Hysom, D., Yeom, J.-S., Gaffney, J., et al., 2019. Scalable topological data analysis and visualization for evaluating data-driven models in scientific applications. *IEEE Trans. Vis. Comput. Graph.* 26, 291–300.
- Lukasczyk, J., Will, M., Wetzels, F., Weber, G.H., Garth, C., 2023. Extreem: scalable augmented merge tree computation via extremum graphs. *IEEE Trans. Vis. Comput. Graph.* 30, 1085–1094.
- Maack, R.G.C., Lukasczyk, J., Tierny, J., Hagen, H., Maciejewski, R., Garth, C., 2024. Parallel computation of piecewise linear morse-smale segmentations. *IEEE Trans. Vis. Comput. Graphics* 30, 1942–1955. <https://doi.org/10.1109/TVCG.2023.3261981>
- Milnor, J.W., 1963. *Morse Theory*. 51, Princeton University Press.
- Morse, M., 1934. *The Calculus of Variations in the Large*, vol. 18. American Mathematical Soc.
- Ophelders, T., Schenfish, A., Sonke, W., Speckmann, B., 2025. Computing geomorphologically salient networks via discrete morse theory. In: 41st International Symposium on Computational Geometry (SoCG 2025), Dagstuhl, Germany, pp. 70:1–70:17.
- Peterka, T., Ross, R., Gyulassy, A., Pascucci, V., Kendall, W., Shen, H.-W., Lee, T.-Y., Chaudhuri, A., 2011. Scalable parallel building blocks for custom data analysis. In: 2011 IEEE Symposium on Large Data Analysis and Visualization. IEEE, pp. 105–112.
- Shivashankar, N., Natarajan, V., 2017. Efficient software for programmable visual analysis using morse-smale complexes. In: *Topological Methods in Data Analysis and Visualization IV: Theory, Algorithms, and Applications VI*. Springer, pp. 317–331.
- Shivashankar, N., Senthilnathan, M., Natarajan, V., 2011. Parallel computation of 2d morse-smale complexes. *IEEE Trans. Vis. Comput. Graph.* 18, 1757–1770.
- Subhash, V., Pandey, K., Natarajan, V., 2020. GPU parallel computation of morse-smale complexes. In: 2020 IEEE Visualization Conference (VIS). IEEE, pp. 36–40.
- Subhash, V., Pandey, K., Natarajan, V., 2022. A GPU parallel algorithm for computing morse-smale complexes. *IEEE Trans. Vis. Comput. Graph.* 29, 3873–3887.
- Thanh, S.L., Ankele, M., Weinkauff, T., 2024. Revisiting accurate geometry for morse-smale complexes. In: 2024 IEEE Topological Data Analysis and Visualization (TopoInVis), pp. 34–43.
- Tierny, J., Pascucci, V., 2012. Generalized topological simplification of scalar fields on surfaces. *IEEE Trans. Vis. Comput. Graph.* 18, 2005–2013.
- Weinkauff, T., Theisel, H., Van Gelder, A., Pang, A., 2010. Stable feature flow fields. *IEEE Trans. Vis. Comput. Graph.* 17, 770–780.